DataForce — Milestone 1 build report

Status: Milestone 1 ("jalon 1") is complete. Every part was built, then independently reviewed for security and statistical correctness, then fixed and re-verified. 408 automated tests pass. **Date:** 17 October 2026 · **Source of truth:** `docs/DECISIONS.md` (every decision and its reason) and `docs/ARCHITECTURE.md` (the build contract). This PDF is generated from `docs/BUILD-REPORT.md` by `docs/tools/build_pdf.py`.

1. In one page

What DataForce is. An analysis workspace for sales and marketing teams. Instead of exporting their CRM into spreadsheets, a team keeps a *minimal CRM* in DataForce (companies, people, deals, notes) and asks it questions: *"Do inbound leads close better?"*, *"Are enterprise deals slower?"*, *"Who reports to whom at this account?"*. TamTam already has better external data than we could build, so DataForce works one level down, at the CRM level, where TamTam does not operate.

The one rule that shapes everything. DataForce only shows a number as a fact when the evidence is strong enough. Otherwise it says **"not verified"** and gives the reasons (too few deals, interval too wide, not significant once many comparisons are accounted for). The AI never computes numbers; it may only explain numbers the code computed.

What Milestone 1 delivers (engine and data, no user interface yet):

Piece	What it does	Where
Minimal CRM	Workspaces, team members, companies, contacts with "reports to", deals, append-only notes and comments, one-click erasure of a person or company	database + `app/services/`
Statistics engine	Win rate, sales-cycle length and deal size, overall and by group, with 95 % confidence intervals and "not verified" gates; a full scan that compares every group with the rest and controls false discoveries	`app/stats/`
Synthetic demo data	200 realistic deals with effects planted on purpose, a pure-noise twin, and an answer key	`app/demo/`
Proof notebook	Runs everything end to end and shows the engine finds what was planted and stays silent on noise	`notebooks/M1_engine_demo.ipynb`
Command line	`dataforce demo / stats / crm / db …`, every read command can answer in JSON (the future "AI-friendly" access layer)	`app/cli/`
LLM and TamTam layers	Kept from v1, made generic for Milestone 2, security fixes completed; fake providers by default, paid enrichment only on an explicit click	`app/llm/`, `app/enrichment/`

What it proves. The demo dataset has 200 deals.

- **It finds what is there.** The engine verifies all 5 planted effects.
- **It isn't fooled by small groups.** The "referral" trap, 24 deals with a spectacular win rate, stays "not verified".
- **It invents nothing.** There are 0 false findings. On the pure-noise twin, it verifies **nothing** out of 93 comparisons.

We also ran it on 200 fresh random datasets. It is deliberately cautious: about 1 run in 10 misses a real effect, and about 1 run in 100 shows a spurious one.

Not in Milestone 1 (by plan): the web interface, the MCP server, the live TamTam connection, title normalisation and the inferred org chart, meeting-transcript themes (Milestones 2 and 3).

2. How it was built

The pivot. The project started (v1) as a "lookalike" tool that crawled company websites to find prospects similar to existing customers. On 17 October the product owner pivoted to DataForce because TamTam already covers that ground. The v1 code that only served the lookalike idea (crawler, open-data connectors, lookalike scoring, its evaluation and its tests) was **deleted** at the product owner's request. What carries over was kept: the infrastructure, the hardened configuration and database session, the services pattern, the LLM layer and the TamTam MCP client.

The team. A tech lead (Claude Opus) wrote the contracts first — the database model, the shared data types, the configuration and a written specification for each part — then handed each part to its own builder agent (Claude Sonnet, low effort, as requested), with **non-overlapping file ownership** so they could work in parallel without stepping on each other:

Step	Agents (Sonnet, low effort)	Output
Build	foundation (DB, migrations, CRM services, infra), generator (synthetic data), stats (engine), LLM/TamTam clean-up	4 modules with their tests
Integrate	integration	CLI commands, executed notebook, end-to-end tests, README
Ratify	2 security reviewers + 1 independent statistics reviewer, all read-only	security reports SR4 and SR5, statistics ratification (in `docs/`)
Fix	3 fixers (DB/infra, stats/demo, LLM/TamTam)	fixes + regression tests, resolution tables

The tech lead checked every hand-back by re-running lint and the tests rather than trusting the report, and kept `docs/DECISIONS.md` current.

3. Product decisions (and why)

Decision	Why
Not enough evidence → "not verified", always with the reasons, and "not verified" is explicitly *not* "no effect".	A salesperson must be able to rely on whatever is shown as a fact.
The AI never calculates. Numbers come from code (SQL counts, statistics); the AI may only name, explain and cite them.	Numbers must be reproducible and auditable.
Notes and comments are append-only. A correction is a new line pointing at the one it corrects; a delete hides the note but keeps the history. A database trigger enforces it, so it holds even if the application has a bug.	Trust and traceability of what was said about a person or account.
Except GDPR erasure. Erasing a person or company hard-deletes them and everything attached, through an explicit, narrow switch.	The right to erasure takes precedence over history.
Strict minimum on third parties: name, title, company, source. No email, no phone.	Data minimisation (GDPR); TamTam is the place for contact data.
Every table belongs to a workspace.	One team = one workspace; demo datasets live side by side; opening the tool to TamTam clients later needs no schema rewrite.
Fixed pipeline stages (prospecting → qualification → proposal → negotiation → won / lost).	Custom pipelines are out of scope for v1.
Synthetic data, announced as such.	No real personal data during the build; a known answer key lets us *prove* the engine is right.
Paid TamTam enrichment only on an explicit click, with a per-call credit ceiling.	No surprise spending.

4. How the statistics work (plain language)

The three questions. *Win rate* — of the deals that are closed, what share was won? *Sales cycle* — for closed deals, how many days from creation to close (median)? *Deal size* — the median amount in euros. Medians are used because one giant deal should not distort the picture.

Confidence intervals. Every number comes with a range that, 95 times out of 100, contains the true value. With 20 deals the range is wide; with 200 it is narrow. Methods: *Wilson* for a rate (reliable even with few deals), *Newcombe* for the difference between two rates, *bootstrap* for medians (re-sample the deals 2,000 times, with a fixed seed so results are reproducible).

The "not verified" gates (approved by the product owner, all configurable):

A result is "not verified" when…	Default
the group has too few deals	fewer than 30
a win rate rests on too few wins or losses	fewer than 10 of either

the range is too wide	more than ±15 points for a rate, more than ±30 % of the value for a median

Comparing groups. The scan compares each group with *the rest* (e.g. inbound leads vs all other sources) on each question, using Fisher's exact test (rates) and the Mann-Whitney test (days, amounts). Because a scan makes about 90 comparisons at once, some would look "significant" by pure chance; the *Benjamini-Hochberg* correction keeps the expected share of false findings among the reported ones at 5 %. A finding is **verified** only if both sides pass the gates, the corrected test is significant and the range of the difference excludes zero.

Missing data is counted and reported, never turned into a group of its own.

What this means in practice. These are real lines from the demo scan (seed 42):

Group vs rest	Question	Result	Verdict and reason
Inbound leads	win rate	65.4 % vs 25.5 %, difference +39.9 pts [+23.3, +53.6]	**verified**
Enterprise	sales cycle	median 118 vs 42 days, +76 [+58, +92]	**verified**
Outbound leads	win rate	9.8 % vs 59.6 %	not verified: only 6 wins (< 10)
Referral leads	win rate	70.8 % vs 33.3 %	not verified: only 24 deals (< 30), only 7 losses
Mid-market	sales cycle	median 57 vs 40 days	not verified: not significant after correction (q = 0.063)

The outbound line shows the rule at work. The gap is real, because the data was built that way. But it rests on 6 won deals, so DataForce says "not verified" rather than put a fragile number in front of a salesperson. The report always adds that "not verified" does not mean "no effect". It means "not enough evidence yet".

5. The synthetic demo data

The generator builds a realistic small company: about 70 accounts, 430 contacts with full org charts (French and English titles, some missing), 200 deals across owners and stages, notes and comments (including corrections). Everything is invented (`.example` domains, made-up names) and deterministic: the same seed always gives the same data.

Kind	In the demo data
Planted on purpose (the engine must find it)	Inbound leads win far more often than outbound. Enterprise deals take much longer to close than SMB deals. Enterprise deals are much larger than SMB deals.
Planted as a trap (must stay "not verified")	Referral leads: a very high win rate, but fewer than 30 closed deals.
Null (must find nothing)	Industry, region and owner have no effect anywhere. The whole **noise variant** has the same shape and no effect at all.

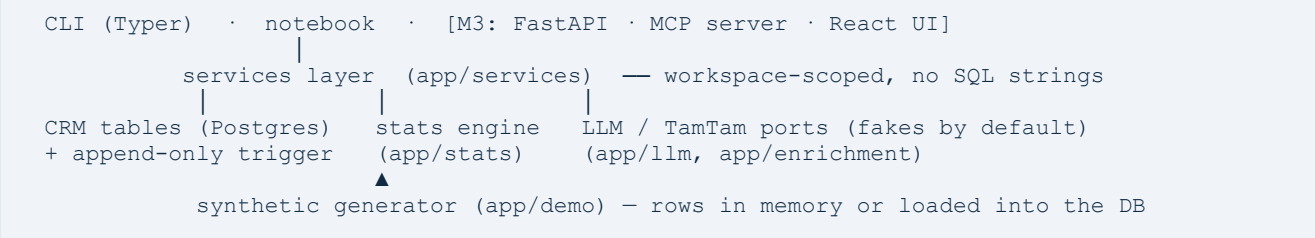
Side effects are expected. Some differences are real consequences of the planted ones without being planted themselves. Small deals close faster because most of them are SMB deals; outbound wins less because inbound wins more. The answer-key checker lists these separately and never counts them as errors. It counts as a **false finding** any verified result in a cell with no planted link, on a small trap group, or in the wrong direction.

How reliable is it? Two independent sweeps of 100 random datasets each, run by a fixer and by the tech lead on different seeds:

Measure	Result
Inbound win-rate effect found	93–96 % of datasets (about 55 inbound deals, close to the minimum)
Enterprise / SMB cycle effects found	99–100 %
Enterprise / SMB amount effects found	97–100 %
Planted datasets with a false finding	0–1 %
Noise datasets with any verified finding	1 %

6. Architecture and structure

Layers. Every way into the system (command line today; web API, MCP server and UI in Milestone 3) goes through one *services* layer, which is the only code that talks to the database. Analyses are pure functions that receive plain rows and return plain JSON-ready results.



Folder structure (backend: about 6,100 lines of application code, 470 lines of migrations, 4,500 lines of tests):

Folder	Contents
`app/domain/`	The contracts: enumerations, shared data types (scan report, gates, ground truth…), provider interfaces
`app/db/`	Database models (9 tables) and the hardened session (timeouts, TLS rules)
`alembic/versions/`	Migrations: schema + append-only trigger, lock-down of public API roles, hardening
`app/services/`	CRM operations (workspaces, users, accounts, contacts + org chart, deals, notes, comments), runs history, analytics, admin
`app/stats/`	Intervals, significance tests, gates, the scan engine
`app/demo/`	Generator, answer-key checker, database loader

`app/llm/`, `app/enrichment/`	Gemini / Anthropic / fake providers, untrusted-text guard, quote verification; TamTam MCP client and fake
`app/cli/`	The `dataforce` command
`tests/`	Unit, database, end-to-end and (opt-in) live tests
`notebooks/`	The executed Milestone 1 notebook
`docs/`	Decisions, architecture contract, security reviews, statistics ratification, this report

Data model.

Table	Holds	Notes
`workspace`	one team's data, or one demo dataset	everything below belongs to a workspace
`app_user`	sales-team members (deal owners, note authors)	login arrives in Milestone 3
`account`	companies	industry, segment, region, size, source
`contact`	people at a company	name, title, "reports to", source — no email or phone
`deal`	opportunities	amount, stage, lead source, created / closed dates, owner
`note` / `comment`	free text on a person / a company	append-only, corrections chain, soft delete
`run`	history of generations and scans, with the full result	lets any number be re-read and cited
`llm_call`	one line per AI call	usage and cost per workspace

Technology.

Choice	Why
Python 3.13, SQLAlchemy 2 (sync) + psycopg 3	Mature, simple; async only where there is network I/O
PostgreSQL 16 + pgvector (Docker locally, pinned image; Supabase EU later)	One database for records, exact aggregates and later text search
numpy + scipy	Reference implementations of the statistical tests
pydantic 2	Every result is a typed, JSON-serialisable object
Typer + Rich	A readable command line with a `--json` switch everywhere
Gemini (main) / Anthropic (swap), behind one interface	Product-owner choice; fakes keep tests and demos free and offline

TamTam via MCP (remote HTTP, bearer token)	What TamTam's own agent exposes; tools discovered at run time from an allow-list
Hashed lock files (pip-tools)	Reproducible, tamper-evident installs

7. Security

Principles. Secrets only in the environment, never logged or shown in errors. Least data on third parties. Untrusted text (notes, transcripts, enrichment results) is wrapped and sanitised so it can never act as instructions to the AI. Database refuses what the application must never do (editing history, self-reporting contacts). Remote databases must use TLS; development passwords are refused outside development. Supabase's public API roles get no access.

Reviews. Separate reviewer agents carried out each review. They could read the code but not change it, and they had to prove each finding, for example by actually trying to rewrite a note in a test database.

Review	Scope	Findings	Status
SR1 (v1)	Website crawler	not applicable	code deleted at the pivot
SR2 (v1)	LLM layer and TamTam connector	20 items	closed: fixed, moot after the pivot, accepted with a reason, or out of scope. Re-verified item by item in SR5
SR3 (v1)	Database, configuration, supply chain	fixed before the pivot	fixes kept and re-verified in SR4
SR4	CRM database, append-only trigger, workspace isolation, erasure, infrastructure	0 critical, 0 high, 3 medium, 5 low, 4 info	all 3 medium, 3 of 5 low and 3 of 4 info fixed with regression tests; the rest accepted with a reason (table in the report)
SR5	LLM / TamTam re-check, statistics and demo data integrity	0 critical, 0 high, 4 medium, 6 low, 3 info	all 4 medium and 5 of the lows fixed with regression tests; the rest accepted or moved to M2

Main things the reviews tested and confirmed hold:

- Notes cannot be rewritten or hard-deleted, even by direct SQL or by emptying the whole table.
- Erasing a person or company leaves no row behind.
- 20 attempts to reach another workspace's data through the services were all refused.
- No SQL is built from user text.
- Passwords and keys never appear in errors.
- Paid enrichment cannot run without an explicit `True`, and a NaN credit limit is refused.
- Hidden text tricks such as look-alike characters and zero-width characters cannot break out of the "untrusted text" wrapper.
- Duplicated or impossible rows (a deal closed before it was created, a negative amount) can no longer produce a "verified" finding.

Accepted residuals (written down, scheduled):

- Erasure does not scrub free-text mentions of a person in *other* notes, or past scan results. A sweep comes with M2's free-text analysis.
- Workspace boundaries are enforced in the services, not yet by composite database keys (M3 option).
- The app should run under a non-owner database role in production (M3).
- A verified quote proves the words exist in the source, not that they support the claim. The M2 interface will show the context.

8. Verification results

Check	Result
Automated tests	**408 pass**: database and config 120, CRM services 21, statistics 43, demo data 39, LLM 85, TamTam connector 90, end-to-end command line 10. Live tests against real AI providers are opt-in and were not run, because no key is needed for M1
Code style and lint (ruff)	clean on the whole backend
Migrations vs models (`alembic check`)	no difference; dev database at `0003_hardening`
Notebook	re-executed after the fix pass, saved with outputs, no errors, no secret or database address in the outputs
Statistics ratification	**RATIFIED**. Every method matches an independent re-implementation to about 16 decimal places, with Newcombe's published example reproduced. The 6 issues raised are all fixed (`docs/review/M1-stats-ratification.md`)
Security	SR4 and SR5 resolution tables show every medium finding fixed (section 7)

Caught during the verification itself:

- **The tech lead's own claim was wrong.** "About 3 % of datasets fail" came from a checker that was too lenient. The real figure is about 10 %, almost all of them cautious misses, and the decision log now says so.
- **The statistics reviewer found two bugs.** The generator crashed on 2 seeds out of 300, and the comparison gates were looser than the rule the product owner approved.
- **The final notebook run exposed a rounding bug.** A confidence interval sat 0.00000000000000006 above its own estimate. It was fixed, with a test.

9. How to run it

Prerequisites: Docker and Python 3.13. Full instructions are in `README.md`, section *Run Milestone 1*.

```
make db-up                                  # local Postgres + pgvector on 127.0.0.1:5433
cd backend && python3.13 -m venv .venv
.venv/bin/pip install --require-hashes -r requirements-dev.lock
.venv/bin/pip install --no-deps -e .
.venv/bin/dataforce db upgrade
.venv/bin/dataforce demo generate --variant planted --seed 42 --replace
.venv/bin/dataforce demo check    --variant planted --seed 42
```

```
.venv/bin/dataforce stats scan    --workspace demo-planted-42
```

10. What comes next

Milestone	Content
M2 (week 2)	Title normalisation (FR/EN dictionary, AI only for unknown titles, cached and flagged); org-chart rules with their evidence; transcript import (PDF / Markdown) and meeting themes counted by code, named by the AI with verified quotes
M3 (week 3)	Minimal React interface, MCP server and API (exact results for AI agents), live TamTam connector (fake by default), authentication, hosted demo

If time runs short: cut meeting analysis first; keep the org chart (it complements TamTam).

Open questions for the product owner.

1. **CRM scope.** Is the minimal CRM (companies, contacts, deals, notes, comments, fixed stages) enough? The product note said "to confirm".
2. **TamTam access.** We need a real MCP account and token, its tool list, and its credit prices, to replace the current assumptions (mobile ≈ 10 credits, email ≈ 1).
3. **AI key.** Add `GEMINI_API_KEY` to `backend/.env` yourself before M2; it is not needed for M1. Never paste it in a chat.
4. **Demo size.** With 200 deals the engine misses the inbound effect about 1 time in 10. A 300–400-deal demo would make it near-certain, at the cost of a less "small team" feel. 200 was the plan.
5. **Version control.** Nothing has been committed yet. Say the word and the build will be committed in logical steps.

Glossary

Term	Meaning
Confidence interval (CI)	The range that most likely contains the true value; narrower with more data
Gate	A minimum-evidence rule; failing one makes a result "not verified"
FDR (false discovery rate)	The expected share of reported findings that are false; held at 5 %
Planted effect	A difference written into the synthetic data on purpose, so we can check the engine finds it
Null dimension	An attribute with no effect at all; the engine must not report one
Append-only	Text that can be corrected or hidden but never silently rewritten
Workspace	One team's isolated data
MCP	Model Context Protocol: the standard way AI agents call tools; TamTam exposes one, DataForce will too